



Kafka Broker Configuration and Performance Tuning

A comprehensive instructor-led training for Kafka administrators, platform engineers, and SREs responsible for operating and optimising Apache Kafka clusters in production environments.

APACHE KAFKA 4.3

INSTRUCTOR-LED TRAINING

65 MODULES

Why Broker Configuration Directly Affects Reliability and Throughput



Configuration Is Not Optional Tuning

Default Kafka settings are designed for basic functionality, not production-grade SLAs. A single misaligned parameter can degrade throughput by 50% or trigger cascading failures across the cluster.

Reliability Levers Are in the Broker

Replication factors, ISR settings, and ack semantics are broker-controlled. Incorrect values silently expose data to loss even when producers report success.

Performance Amplification Effect

In a six-broker cluster, a misconfiguration that adds 10 ms latency per request compounds across partitions, leaders, and replication, becoming visible SLA violations at scale.

- ❑ Every parameter covered in this training maps directly to an observable production metric. Configuration decisions must be justified by data.

Learning Objectives

By the end of this course, participants will be able to:

01

Explain Broker Architecture

Describe how a Kafka broker processes produce and fetch requests, manages the page cache, and maintains replicas through its internal thread model.

02

Build a Production Configuration Baseline

Configure listeners, log directories, and dynamic settings using KRaft for a six-broker, three-controller cluster.

03

Tune Storage, Network and Replication

Apply segment, retention, thread, and replica-fetcher settings using evidence-based reasoning and workload measurement.

04

Diagnose and Remediate Production Issues

Follow structured troubleshooting workflows for high produce latency, disk growth, and ISR shrink events.

05

Apply a Safe Tuning Workflow

Use baseline → hypothesis → canary → compare → rollback methodology to change configuration without service disruption.

Our Running Production Environment

All examples, commands, and calculations in this training reference the following reference cluster. Configurations will be validated against this topology throughout every module.

6 Brokers

IDs 1-6, spread across three availability zones, two brokers per zone.

3 KRaft Controllers

Dedicated controller nodes (IDs 101-103), one per AZ. No ZooKeeper.

RF = 3

All topics replicated three times. `min.insync.replicas=2` enforced cluster-wide.

Mixed Workload

High-throughput event streams, low-latency transactional topics, and long-retention audit topics co-hosted on the same cluster.

 This environment is deliberately representative. Workload characteristics will vary in your organisation — always validate settings against your own profiling data.

Performance Tuning Principle: Measure Before Changing

The Cardinal Rule

Never adjust a configuration parameter without first establishing a measurable baseline. Changing multiple settings simultaneously makes root-cause analysis impossible.

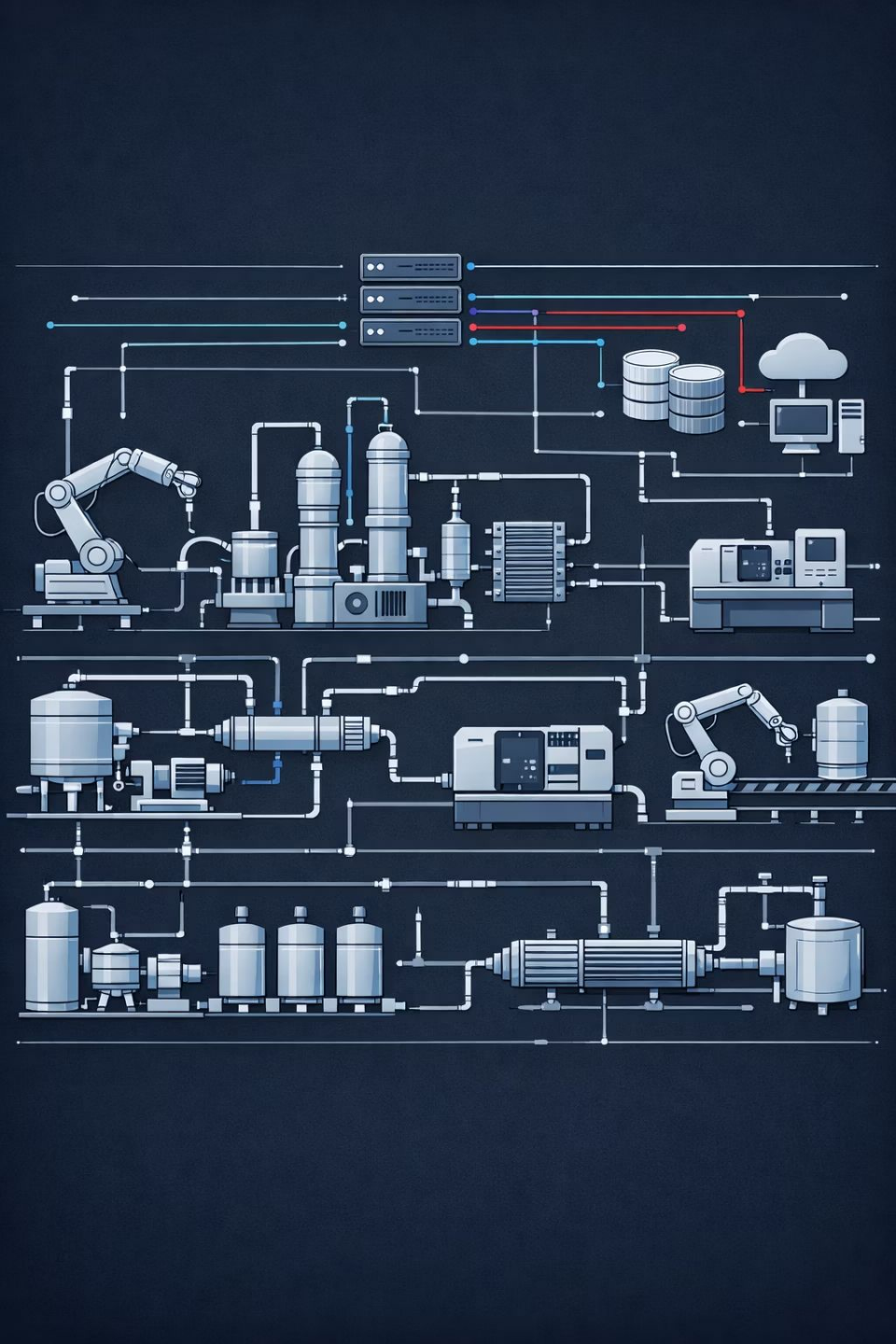
One change. One measurement cycle. Document results before the next iteration.

What to Measure First

- Producer request latency (p99) and throughput (MB/s)
- Fetch consumer lag and end-to-end latency
- Request queue depth and handler idle ratio
- Disk utilisation, I/O await and page-cache hit rate
- ISR shrink events and replica lag metrics

Safe Change Cadence

Apply on one canary broker → observe for one full traffic cycle → compare against baseline → promote or rollback.



Module 1

How a Kafka Broker Processes Work

Understanding the internal mechanics of a broker — from network acceptance to disk write — is the foundation for every tuning decision in this course.

MODULE 1 OF 7

BROKER ARCHITECTURE

Broker Responsibilities: Requests, Logs, Replicas and Metadata



Request Handling

Accepts produce, fetch, metadata, and admin requests from clients and other brokers. Routes each request to the correct partition leader.



Log Management

Writes producer data to partition log segments on disk. Manages segment rolling, retention enforcement, and index maintenance.



Replica Coordination

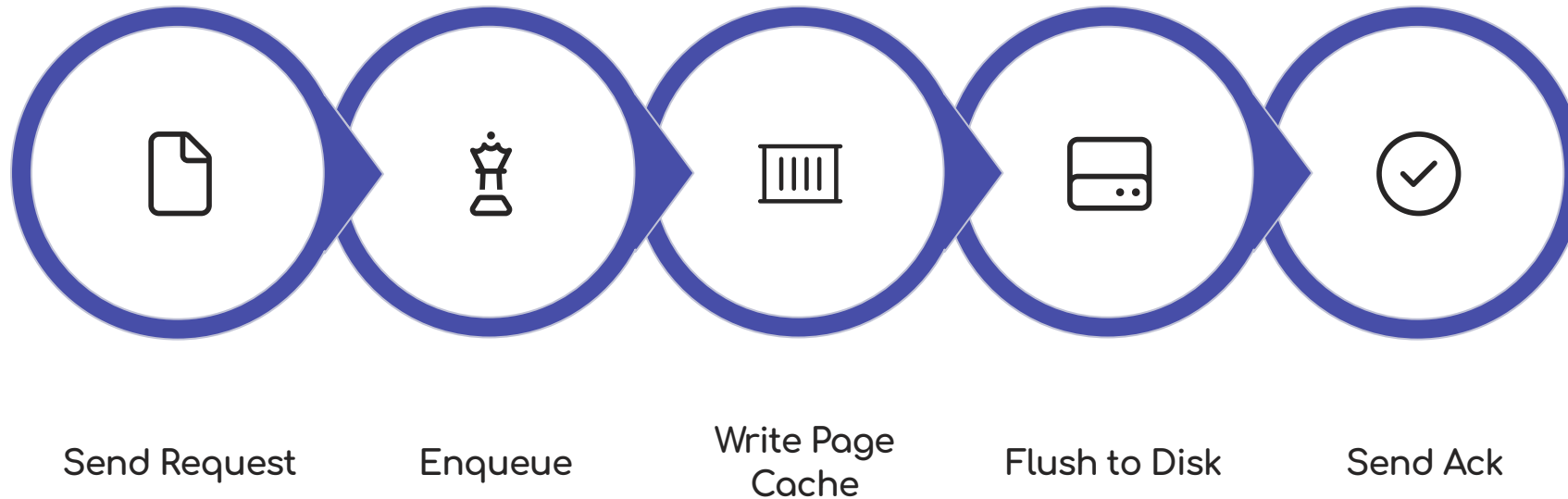
Leaders track ISR membership. Followers fetch from leaders and advance the high watermark. Brokers handle leader election in KRaft mode.



Metadata Propagation

In KRaft, brokers receive metadata updates from controllers via the metadata log. Brokers serve cluster metadata to clients on request.

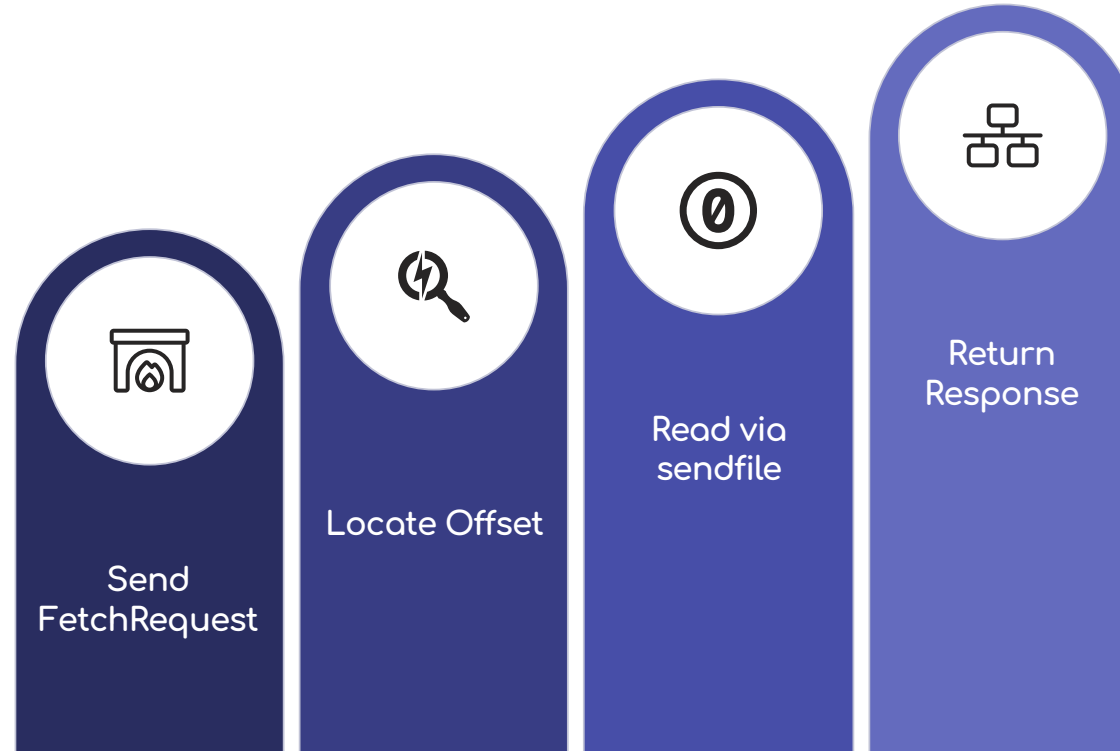
Produce Request Journey: Client to Partition Log



The produce path is entirely sequential within a partition. Network threads receive and enqueue; I/O threads validate, write to the OS page cache, and generate acknowledgements. The OS manages actual disk flush. Understanding this pipeline is critical before tuning thread counts or flush intervals.

- ❏ Administrator takeaway: Producer latency is dominated by the request queue wait time and the ack=all replication round-trip — not the local disk write.

Fetch Request Journey: Log to Consumer



Kafka's fetch path relies heavily on the OS page cache and zero-copy I/O (sendfile syscall). When the requested data is hot in cache, disk is never touched. Cache misses cause significant latency spikes. Consumers reading behind the write frontier will cause cache eviction that degrades producer performance.

- ❑ Administrator takeaway: Consumer lag is not only a consumer problem — lagging consumers evict hot pages and raise produce latency for all partitions on that broker.

Network Threads, Request Queues and I/O Threads

Kafka's request-processing model uses two distinct thread pools separated by a bounded queue. Understanding the division of labour helps identify whether a bottleneck is in the network or compute layer.

Component	Responsibility	Key Config
Network Threads	Accept connections, read request bytes, write response bytes	num.network.threads
Request Queue	Buffer between network and I/O layers; bounded to prevent unbounded memory use	queued.max.requests
I/O Threads	Decode requests, interact with logs, produce responses	num.io.threads
Response Queue	Per-network-thread queue holding completed responses awaiting transmission	(internal, bounded)

Page Cache and Sequential Disk I/O

Why the Page Cache Is Central

Kafka does not maintain its own read cache. It relies entirely on the Linux page cache. Producers write to the cache; consumers read from it via zero-copy. Physical disk I/O occurs only when the OS decides to flush dirty pages or when a cache miss forces a read.

Sequential I/O Advantages

- Append-only writes maximise disk throughput on both HDD and SSD
- Read-ahead prefetch fills cache for consumer fetch patterns
- Avoiding random I/O eliminates seek latency on spinning disks

Administrator Implication

Heap size should be kept small (6–8 GB recommended) to leave maximum RAM available for the page cache. Swapping is catastrophic for broker performance.

Replica Fetchers and ISR Maintenance

Follower Fetch Loop

Each follower broker runs one or more `ReplicaFetcherThread` per leader broker. These threads issue `FetchRequests` at `replica.fetch.min.bytes` and `replica.fetch.wait.max.ms` boundaries, advancing the local log end offset.

ISR Membership Criteria

A replica remains In-Sync if its lag behind the leader stays within `replica.lag.time.max.ms` (default 30 s). Exceeding this threshold triggers an ISR shrink event, reducing the pool available for `acks=all` commits.

High Watermark Advancement

The leader advances the High Watermark (HW) only when all ISR replicas have fetched up to that offset. Consumers can only read up to the HW, providing read-your-writes consistency guarantees.

KRaft: Broker and Controller Responsibilities

Controller Node Role

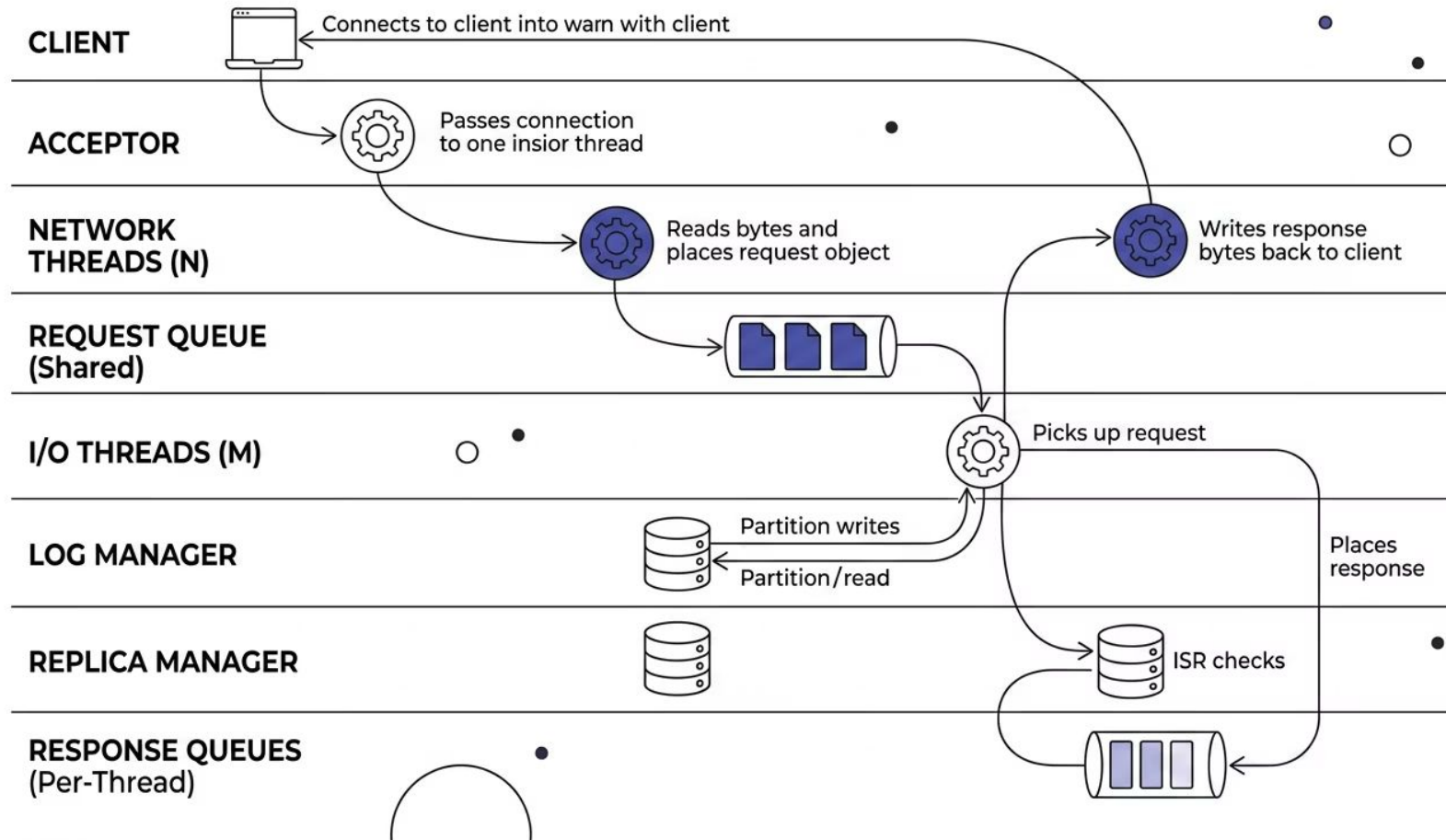
- Runs the Raft consensus algorithm across three dedicated controller nodes
- Maintains the authoritative metadata log (cluster topology, partition assignments, configs)
- Elects partition leaders and propagates changes to broker nodes
- One active controller, two standby controllers (IDs 101-103 in our reference cluster)

Broker Node Role

- Serves all client traffic — produce, fetch, metadata, admin
- Receives metadata updates from the active controller via the metadata log
- Does not participate in Raft consensus; cannot become active controller
- Configured with `process.roles=broker` and the `controller.quorum.voters` bootstrap list

 In KRaft mode, ZooKeeper is entirely absent. The metadata log replaces it, enabling faster failover (typically under 30 seconds) and simpler operational topology.

Complete Broker Request-Path Diagram



This end-to-end path defines the two tuning axes: network thread count governs connection and byte-transfer capacity; I/O thread count governs request processing and disk interaction throughput. The bounded request queue provides natural backpressure.

Knowledge Check: Locate the Likely Bottleneck

Consider these observed metrics on broker-3 in our six-broker cluster. Identify where the bottleneck lies before looking at the answer below.

Observation A

Request handler idle ratio: 8%.
The I/O thread pool is nearly saturated. Network thread idle ratio remains at 65%.

Observation B

Request queue depth: 48 / 500.
Queue is not full. Requests are entering faster than they can be processed.

Observation C

Disk I/O await: 2 ms. Disk is healthy. Page cache hit rate is above 95%.

- ❏ Answer: The bottleneck is the I/O thread pool (`num.io.threads`). Disk is healthy; network is idle; only the handler threads are saturated. Increasing `num.io.threads` is the correct hypothesis to test — measure after applying on canary broker-3 first.

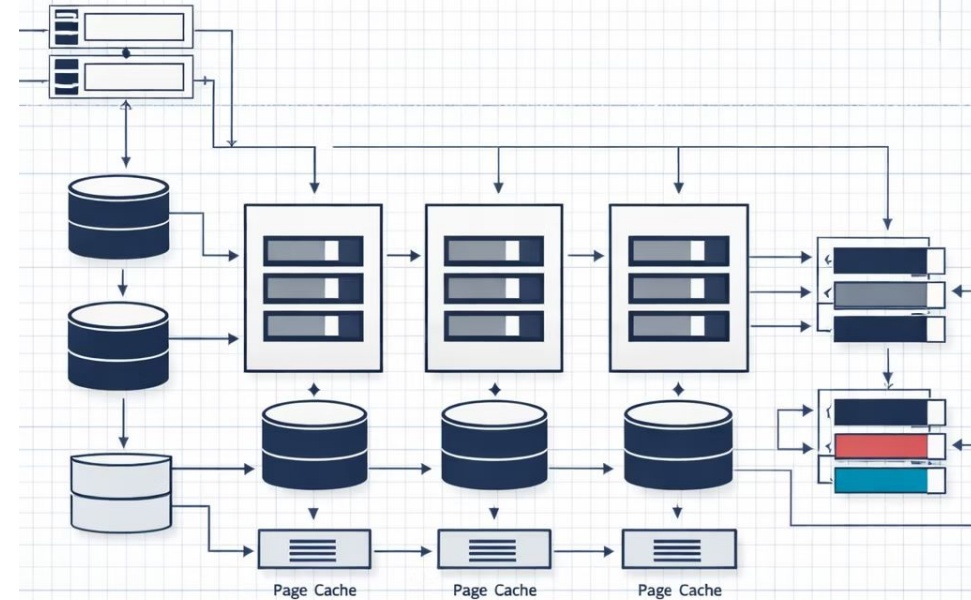
Module 2

Building a Production Broker Baseline

Establishing correct foundational configuration before tuning performance parameters. A solid baseline prevents the majority of production incidents.

MODULE 2 OF 7

PRODUCTION CONFIGURATION



node.id, process.roles and Controller Bootstrap

These three parameters are the identity foundation of every KRaft broker. They must be set correctly before the node starts for the first time and cannot be changed without wiping the node's state.

```
# Broker node (e.g., broker-1)
node.id=1
process.roles=broker
controller.quorum.voters=101@ctrl-1:9093,102@ctrl-2:9093,103@ctrl-3:9093

# Controller node (e.g., controller-101)
node.id=101
process.roles=controller
controller.quorum.voters=101@ctrl-1:9093,102@ctrl-2:9093,103@ctrl-3:9093
```

node.id must be unique across the entire cluster. process.roles determines whether the node handles client traffic, consensus, or both. controller.quorum.voters lists all controller participants.

listeners vs. advertised.listeners

listeners

Defines the socket addresses the broker binds to on the host network interface. The broker listens for incoming connections on these addresses.

Example:

```
listeners=PLAINTEXT://0.0.0.0:9092,INTERNAL://0.0.0.0:9093
```

Use 0.0.0.0 to bind to all interfaces, or a specific IP to restrict binding.

advertised.listeners

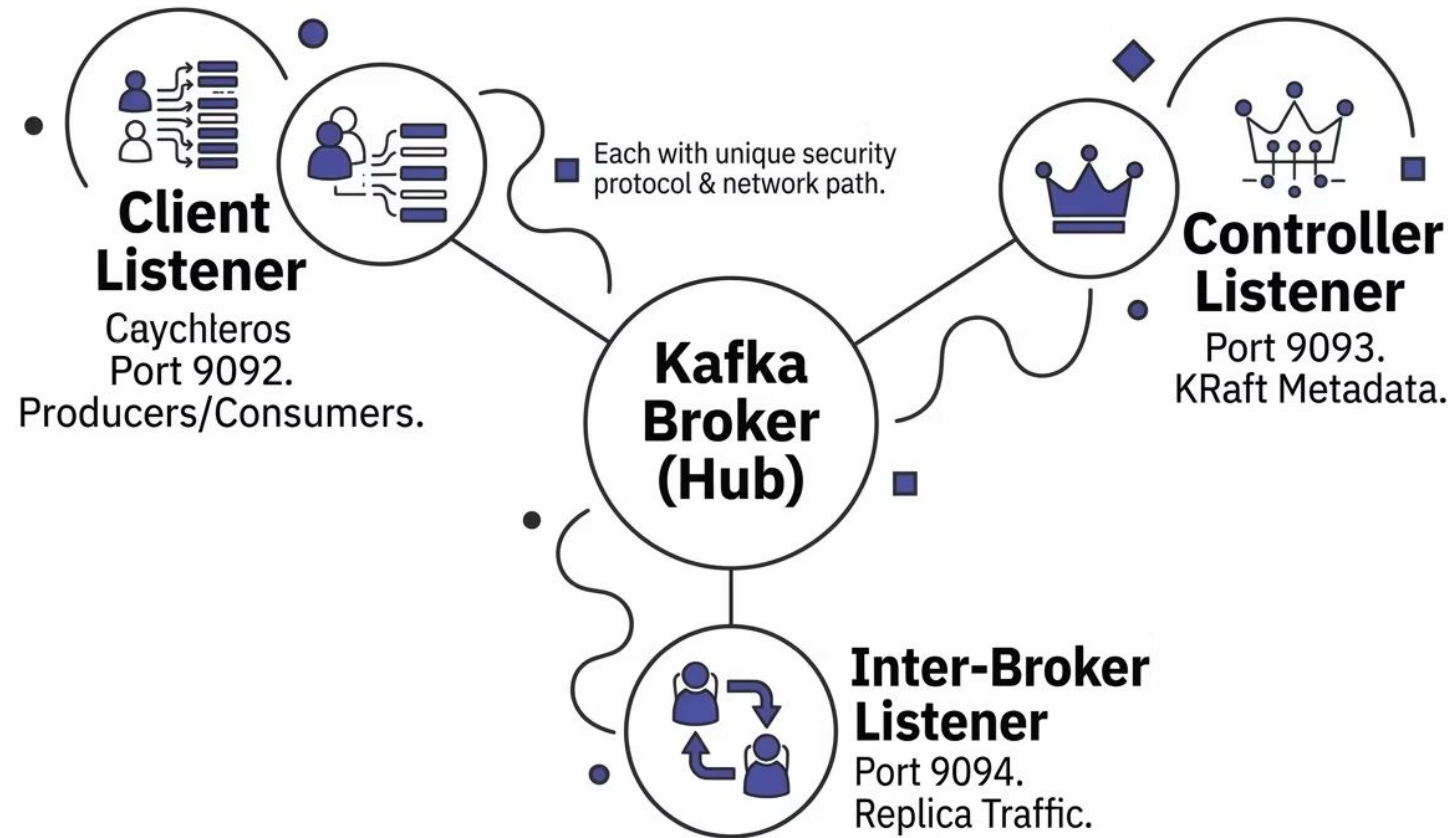
Defines the addresses published to clients in metadata responses. Clients use these addresses to connect after discovering the broker.

Example: `advertised.listeners=PLAINTEXT://broker-1.kafka.internal:9092,INTERNAL://broker-1.internal:9093`

Must be routable from client networks. Critical in containerised and cloud deployments where internal bind addresses differ from external addresses.

 A mismatch between listeners and advertised.listeners is the most common cause of client connection failures after cluster reconfiguration or migration.

Client, Inter-Broker and Controller Listener Design



Separating listener purposes onto distinct ports allows independent security policies, network ACLs, and monitoring. Client traffic and replication traffic should never share a single listener in production to prevent resource contention and to simplify firewall rule management.

listener.security.protocol.map and Secure Communication

Maps each named listener to its security protocol. This single parameter controls whether a listener uses plaintext, TLS, SASL, or SASL+TLS.

```
listener.security.protocol.map=\
```

```
CLIENT:SASL_SSL,\
```

```
INTERNAL:SSL,\
```

```
CONTROLLER:SSL
```

```
inter.broker.listener.name=INTERNAL
```

```
sasl.mechanism.inter.broker.protocol=PLAIN
```

CLIENT → SASL_SSL

Authenticates external producers and consumers with SASL; encrypts with TLS.

INTERNAL → SSL

Encrypts replica fetch and inter-broker traffic; mutual TLS recommended.

CONTROLLER → SSL

Encrypts KRaft metadata log replication between controllers and brokers.

log.dirs and metadata.log.dir

log.dirs — Partition Data

- Comma-separated list of directories where partition log segments are stored
- Multiple paths enable JBOD usage across several physical disks
- New partitions are assigned to the directory with the fewest partitions (not by free space)
- Example: `log.dirs=/data/kafka-1,/data/kafka-2,/data/kafka-3`

metadata.log.dir — KRaft State

- Separate directory for the KRaft metadata log
- Should reside on a dedicated, high-reliability disk — distinct from partition data
- Metadata log corruption can require a full cluster restore
- Example: `metadata.log.dir=/data/kraft-metadata`

⊗ Never place `metadata.log.dir` inside one of the `log.dirs` paths. Mix-up causes KRaft bootstrap failures that require manual recovery.

Static, Per-Broker and Cluster-Wide Dynamic Settings

Kafka configuration properties fall into three mutability categories. Understanding these prevents failed dynamic updates and unnecessary broker restarts.

Category	Definition	Example	Restart Required?
Static	Set only in <code>server.properties</code> ; cannot be changed at runtime	<code>node.id</code> , <code>process.roles</code> , <code>listeners</code>	Yes
Per-Broker Dynamic	Set per node via <code>kafka-configs</code> ; overrides static value for that broker only	<code>num.io.threads</code> , <code>log.flush.interval.ms</code>	No
Cluster-Wide Dynamic	Set once; applies to all brokers unless overridden at broker level	<code>log.retention.ms</code> , <code>min.insync.replicas</code>	No

Read-Only, Per-Broker and Cluster-Wide Update Modes

The Kafka documentation classifies each property by its allowed update mode. Check the documentation before attempting a dynamic change to avoid silent failures.

→ read-only

Property can only be changed in `server.properties` and requires a broker restart. Attempting to set via `kafka-configs.sh` returns an error. Example: `log.dirs`, `node.id`.

→ per-broker

Property can be changed at runtime for a specific broker using `--entity-type brokers --entity-name <id>`. The value stored in the metadata log overrides the static file for that node only.

→ cluster-wide

Property applies to all brokers when set with `--entity-type brokers --entity-name <default>`. Individual per-broker overrides still take precedence.

Inspect Effective Broker Configurations

Always verify the active configuration rather than assuming the file contents are in effect. Dynamic overrides may differ from `server.properties`.

```
# Describe all dynamic overrides for broker 1
kafka-configs.sh \
  --bootstrap-server broker-1:9092 \
  --describe \
  --entity-type brokers \
  --entity-name 1

# Describe cluster-wide (default) dynamic overrides
kafka-configs.sh \
  --bootstrap-server broker-1:9092 \
  --describe \
  --entity-type brokers \
  --entity-name <default>
```

Output shows only properties with active dynamic overrides. Properties not listed are governed by `server.properties` or Kafka defaults. Combine both sources to understand the full effective configuration.

Add and Remove a Dynamic Configuration Override

```
# Add a per-broker override: increase I/O threads on broker-3 (canary)
```

```
kafka-configs.sh \  
  --bootstrap-server broker-1:9092 \  
  --alter \  
  --entity-type brokers \  
  --entity-name 3 \  
  --add-config num.io.threads=16
```

```
# Remove the override, reverting to static file value
```

```
kafka-configs.sh \  
  --bootstrap-server broker-1:9092 \  
  --alter \  
  --entity-type brokers \  
  --entity-name 3 \  
  --delete-config num.io.threads
```

Changes take effect immediately without restart. Always use `--describe` after altering to confirm the override is stored. Deleting an override does not restart the broker; the static value is applied on the next poll cycle (within seconds).

Annotated Production server.properties Example

```
# Identity (static — restart required to change)
node.id=1
process.roles=broker
controller.quorum.voters=101@ctrl-1:9093,102@ctrl-2:9093,103@ctrl-3:9093

# Listeners (static)
listeners=CLIENT://0.0.0.0:9092,INTERNAL://0.0.0.0:9094,CONTROLLER://0.0.0.0:9093
advertised.listeners=CLIENT://broker-1.kafka.internal:9092,INTERNAL://broker-1.internal:9094
listener.security.protocol.map=CLIENT:SASL_SSL,INTERNAL:SSL,CONTROLLER:SSL
inter.broker.listener.name=INTERNAL

# Storage (static)
log.dirs=/data/kafka-1,/data/kafka-2,/data/kafka-3
metadata.log.dir=/data/kraft-metadata

# Replication baseline (cluster-wide dynamic)
default.replication.factor=3
min.insync.replicas=2
```

Parameters are grouped by function. Static parameters appear at the top; dynamic candidates are managed via `kafka-configs.sh` at runtime.

Baseline Configuration Review Checklist



Identity

node.id unique cluster-wide;
process.roles matches node
purpose; cluster.id consistent on
all nodes.



Listeners

advertised.listeners resolve from
client networks; separate listeners
for client, inter-broker, controller
traffic.



Security

TLS enabled on all listeners; SASL
configured for client listener;
mutual TLS for controller listener.



Storage

log.dirs span separate physical disks; metadata.log.dir
on dedicated path; no overlap with log.dirs.



Replication

default.replication.factor=3; min.insync.replicas=2;
offsets.topic.replication.factor=3.

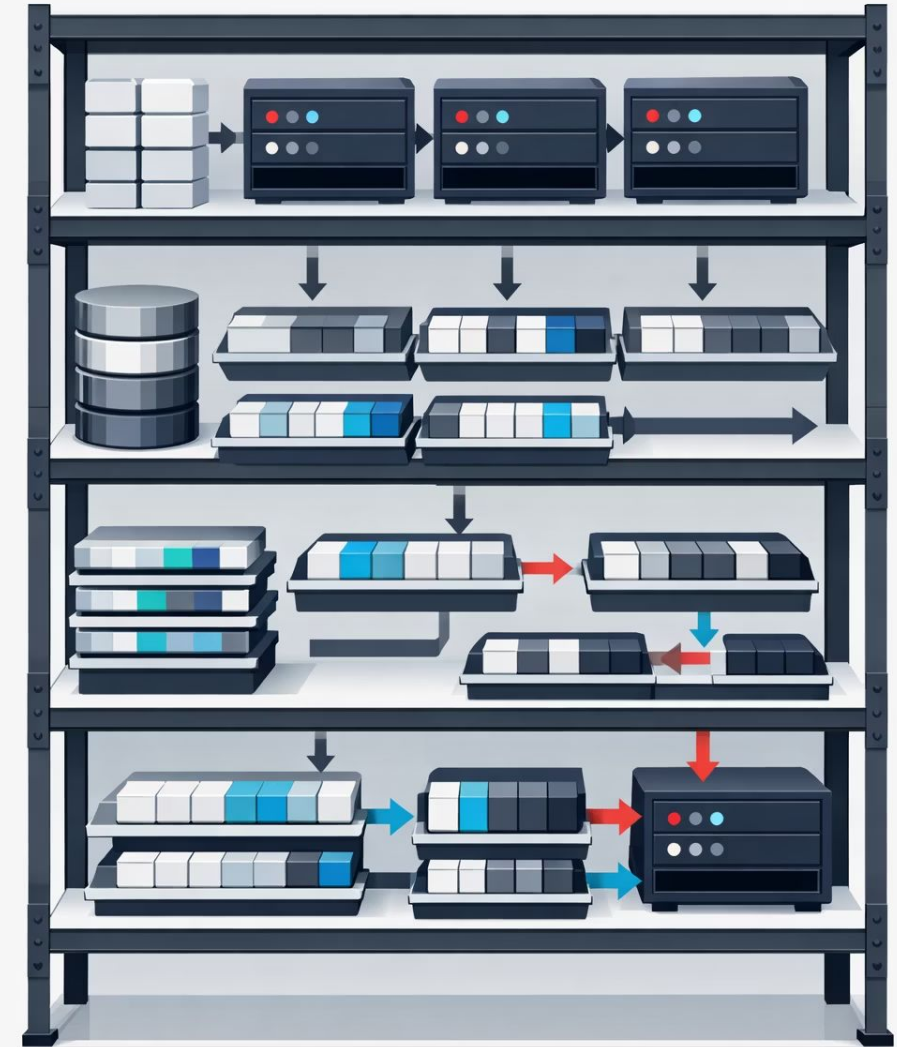
Module 3

Storage, Segments and Retention

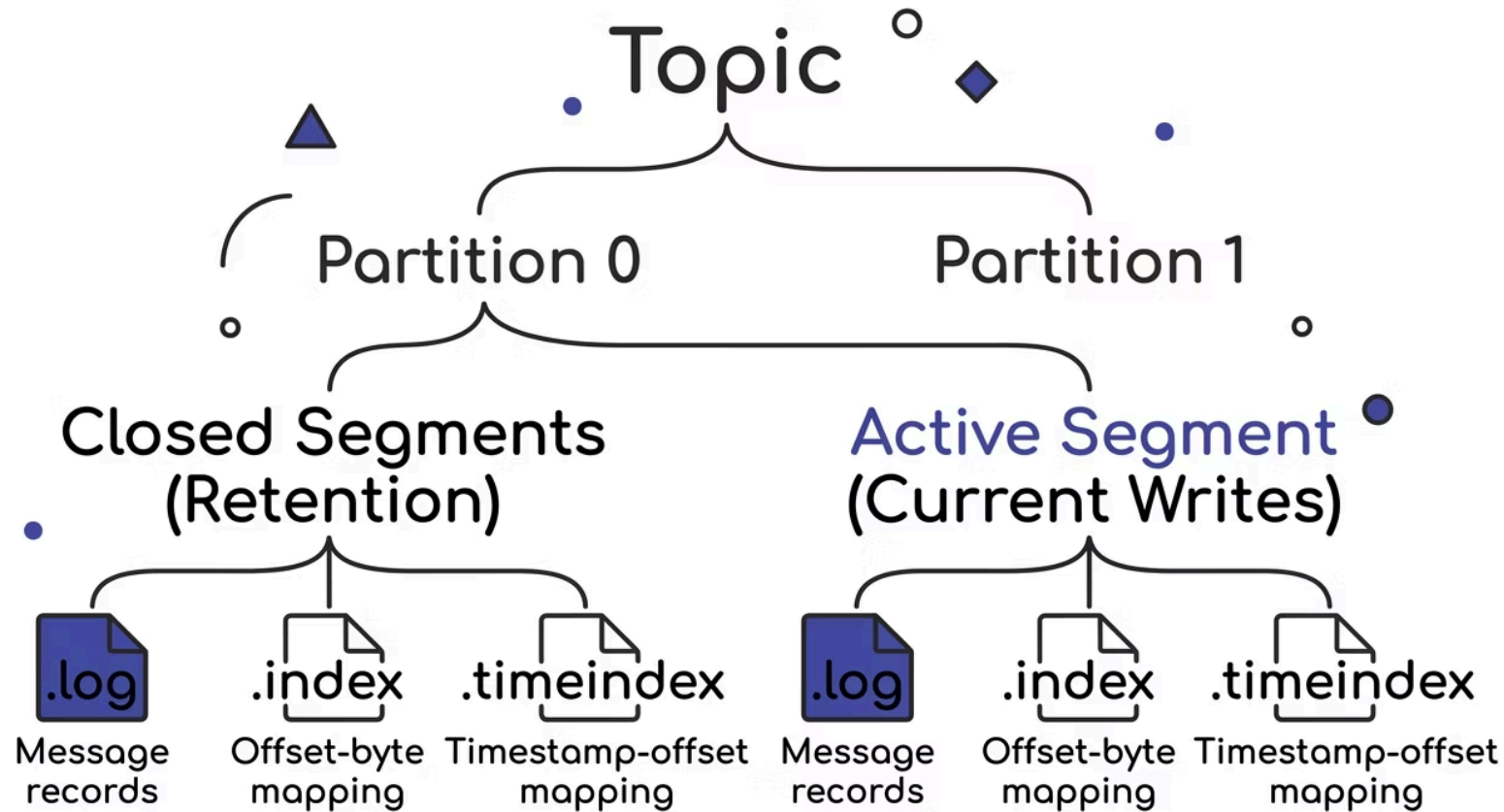
How Kafka organises data on disk, how to choose storage hardware, and how to tune retention and compaction for production workloads.

MODULE 3 OF 7

STORAGE AND LOG TUNING



Kafka Storage Layout: Partitions, Segments and Index Files



The segment is the unit of retention and the unit of compaction. Only closed (inactive) segments are eligible for deletion or compaction. The active segment is always open for appends and is never deleted while the partition leader is running.

Choosing Disks: SSD, HDD, Throughput, Latency and Recovery

Dimension	SSD (NVMe/SATA)	HDD (7200 RPM)
Sequential Write	High (500 MB/s–3 GB/s)	Moderate (100–200 MB/s)
Random I/O	Excellent — benefits compaction	Poor — avoid compaction-heavy workloads
Recovery Speed	Fast replica catch-up	Slower; consider more disks
Cost per TB	Higher	Lower — viable for high-retention topics

- i** Administrator takeaway: SSDs are strongly recommended for low-latency workloads and heavily compacted topics. HDDs remain cost-effective for high-volume, long-retention archive topics where sequential throughput dominates.

Multiple log.dirs and JBOD Implications

How JBOD Works in Kafka

When log.dirs lists multiple paths (one per physical disk), Kafka distributes new partitions across directories using a round-robin least-partitions algorithm. Each disk operates independently — a single disk failure affects only its partitions.

Key Implications for Administrators

- Partition assignment is by count, not by free space — skewed partition sizes can exhaust one disk while others remain empty
- A failed disk triggers LogDirFailureChannel; affected partitions become offline until the disk is recovered or partitions are reassigned
- Monitor per-directory disk usage separately; aggregate metrics mask imbalance
- Prefer uniform disk sizes to avoid unexpected capacity ceilings

log.segment.bytes and log.roll.ms

log.segment.bytes

Controls: Maximum size of an active log segment before it is closed and a new one opened.

Default: 1 GB. Evidence for changing: Producers writing very small batches hold segments open for hours, delaying retention. Very large values delay segment closing and retention enforcement.

Trade-off: Smaller segments increase the number of open file descriptors and index files. Workload testing required.

log.roll.ms

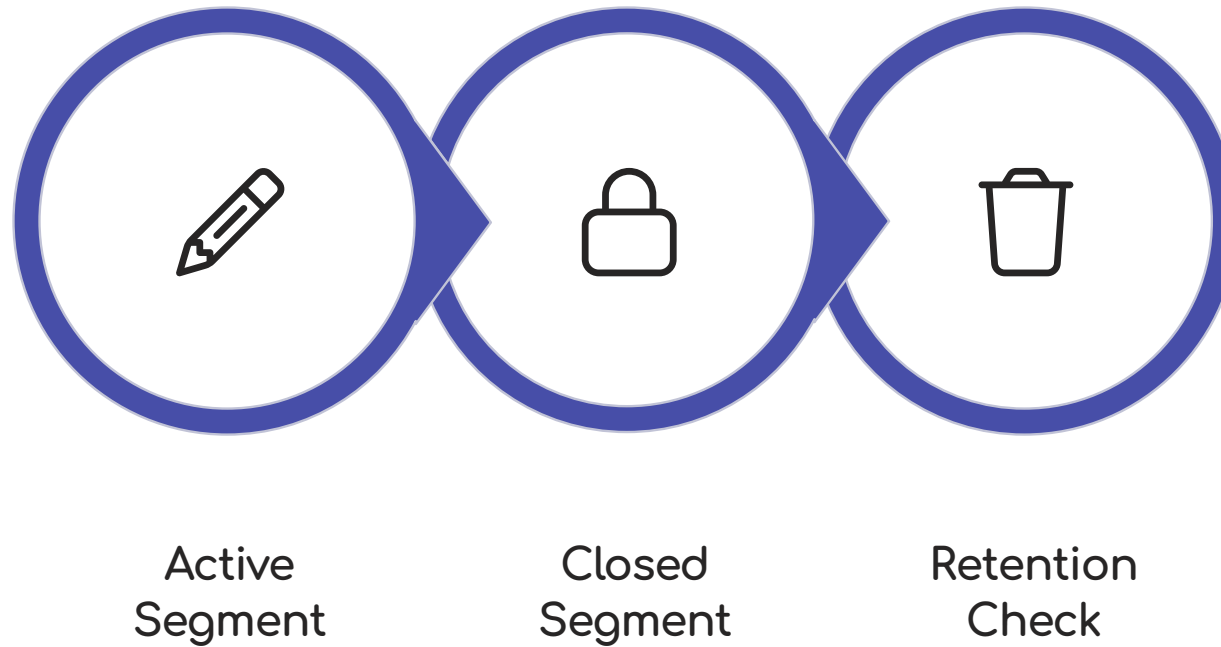
Controls: Maximum time an active segment stays open regardless of size, forcing a time-based roll.

Default: Not set (log.roll.hours=168 applies).

Evidence for changing: Topics with very low produce rates that need timely retention enforcement.

Trade-off: Very short values create many small segments, increasing file handle pressure and index overhead. No restart required (cluster-wide dynamic).

Retention Is Applied to Closed Segments



This lifecycle is frequently misunderstood. Data written one minute ago may reside in the active segment and cannot be deleted, even if retention.ms has elapsed for that data. Retention lag is bounded by segment roll frequency. Tune log.segment.bytes and log.roll.ms together with retention targets to control maximum retention overshoot.

Time-Based and Size-Based Retention

Parameter	Controls	Default	Restart?
log.retention.ms	Delete segments older than this value. Measured by the largest timestamp in the segment.	7 days (604800000 ms)	No
log.retention.bytes	Delete oldest segments when total partition log size exceeds this value.	-1 (unlimited)	No
log.retention.check.interval.ms	How often the log cleaner checks for segments eligible for deletion.	5 minutes	No

- ❏ Both retention.ms and retention.bytes are evaluated; whichever threshold is reached first triggers deletion. Setting retention.bytes is strongly recommended in production to prevent disk exhaustion during unexpected traffic spikes.

Storage-Capacity Planning Formula

Use this formula to estimate required raw disk capacity for a topic before deployment. Apply it to each topic independently, then sum across all topics per broker.

$$\text{Raw Capacity} = \frac{\text{Throughput (MB/s)} \times \text{Retention (s)} \times \text{Replication Factor}}{\text{Brokers}} \times \text{Headroom Factor}$$

Where Headroom Factor accounts for disk overhead, compaction working space, and burst headroom. A value of 1.3–1.5 is typical for production deployments. Throughput should represent peak sustained throughput, not average, to avoid undersizing during traffic spikes.

Worked Calculation: Seven-Day Retention with RF=3

Reference cluster: one topic, 100 MB/s peak produce rate, 7-day retention, RF=3, 6 brokers.

$$\begin{aligned}\text{Raw Capacity} &= \frac{100 \text{ MB/s} \times 604800 \text{ s} \times 3}{6} \times 1.4 \\ &= \frac{100 \times 604800 \times 3}{6} \times 1.4 = \frac{181440000}{6} \times 1.4 = 30240000 \times 1.4 \approx 42.3 \text{ TB total}\end{aligned}$$

Per Broker

≈ 7.1 TB raw per broker across all three log directories.

Per Disk (3 disks)

≈ 2.4 TB per disk. Plan for ≥ 3 TB physical disks to accommodate headroom.

Headroom Factor Used

1.4× for compaction working space, segment overhead and burst tolerance.

Disk Headroom for Failures, Reassignments and Compaction



Broker Failure Headroom

When a broker fails, its partitions reassign to surviving brokers. Remaining brokers temporarily absorb extra replicas. Each broker needs at least $(1/N) \times RF$ additional capacity for the reassignment period.



Reassignment Traffic

Partition reassignment copies entire segments between brokers. Source and destination both hold the data during transfer. Plan for up to $2 \times$ partition size during active reassignment.



Compaction Working Space

Log compaction requires space for the dirty buffer and the merged output segment. Topics with high update rates can temporarily double their segment footprint during compaction cycles.

Log Compaction and Cleaner Workload

How Compaction Works

For topics with `cleanup.policy=compact`, the log cleaner retains only the most recent record for each message key. Older records with the same key are removed, reducing storage while preserving the latest state.

Tombstone records (null-value) mark a key for deletion and are themselves retained for `delete.retention.ms` before final removal.

Cleaner Workload Characteristics

- Cleaner threads run continuously in the background, reading dirty segments and writing a merged clean segment
- High key cardinality increases buffer memory pressure on the cleaner
- Topics receiving frequent updates (high dirty ratio) demand more cleaner CPU and I/O
- Compaction does not reduce size to zero — it reduces to unique-key size

Cleaner Tuning: Threads, Buffer and Dirty Ratio

`log.cleaner.threads`

Controls: Number of background threads processing compaction simultaneously.

Default: 1. Evidence: Cleaner falling behind (dirty ratio rising). Benefit: Parallelises compaction across multiple topics. Trade-off: Each thread consumes `log.cleaner.dedupe.buffer.size` memory. No restart required.

`log.cleaner.dedupe.buffer.size`

Controls: Total memory across all cleaner threads for the deduplication hash map.

Default: 128 MB. Evidence: Cleaner efficiency below 50% (high segment re-reads). Benefit: Larger buffer reduces re-scan passes. Trade-off: Consumes heap or off-heap memory. Restart required.

`min.cleanable.dirty.ratio`

Controls: Minimum fraction of log that must be dirty before cleaner engages.

Default: 0.5 (50%). Lowering triggers more frequent compaction runs. Raising reduces I/O overhead. Workload testing required — no universal value.

Why Aggressive Log Flushing Can Reduce Performance

The Temptation

Administrators often set `log.flush.interval.messages` or `log.flush.interval.ms` to very low values believing this improves durability. In practice, Kafka's durability guarantee comes from replication, not from `fsync` frequency.

The Performance Cost

- Frequent `fsync` calls block the I/O thread until the OS confirms the write to physical media
- Blocking an I/O thread stalls all pending requests in the queue for that thread
- With `RF=3` and `acks=all`, a disk `fsync` failure is recoverable — replication provides the durability
- Kafka default: rely entirely on OS page cache flushing (`pdflush` / `dirty_expire_centisecs`)

❏ Administrator takeaway: Leave `log.flush.interval.messages` and `log.flush.interval.ms` at their defaults (OS-managed) unless your compliance requirements explicitly mandate synchronous `fsync`. Validate with your security and compliance team.

Storage Symptoms, Evidence and Corrective Actions

Symptom	Evidence / Metric	Safe Action
Disk filling faster than expected	kafka.log:type=LogSize growing; retention.bytes not set	Set log.retention.bytes per partition; verify segment roll frequency
Retention not removing old data	Segments older than retention.ms still present on disk	Confirm segment is closed; check log.retention.check.interval.ms
Compaction falling behind	kafka.log:type=MaxDirtyPercent rising; cleaner-stats in JMX	Increase log.cleaner.threads; increase dedupe buffer size
High disk I/O await	iostat: await > 10 ms; iowait CPU elevated	Check for compaction-write amplification; consider SSD upgrade

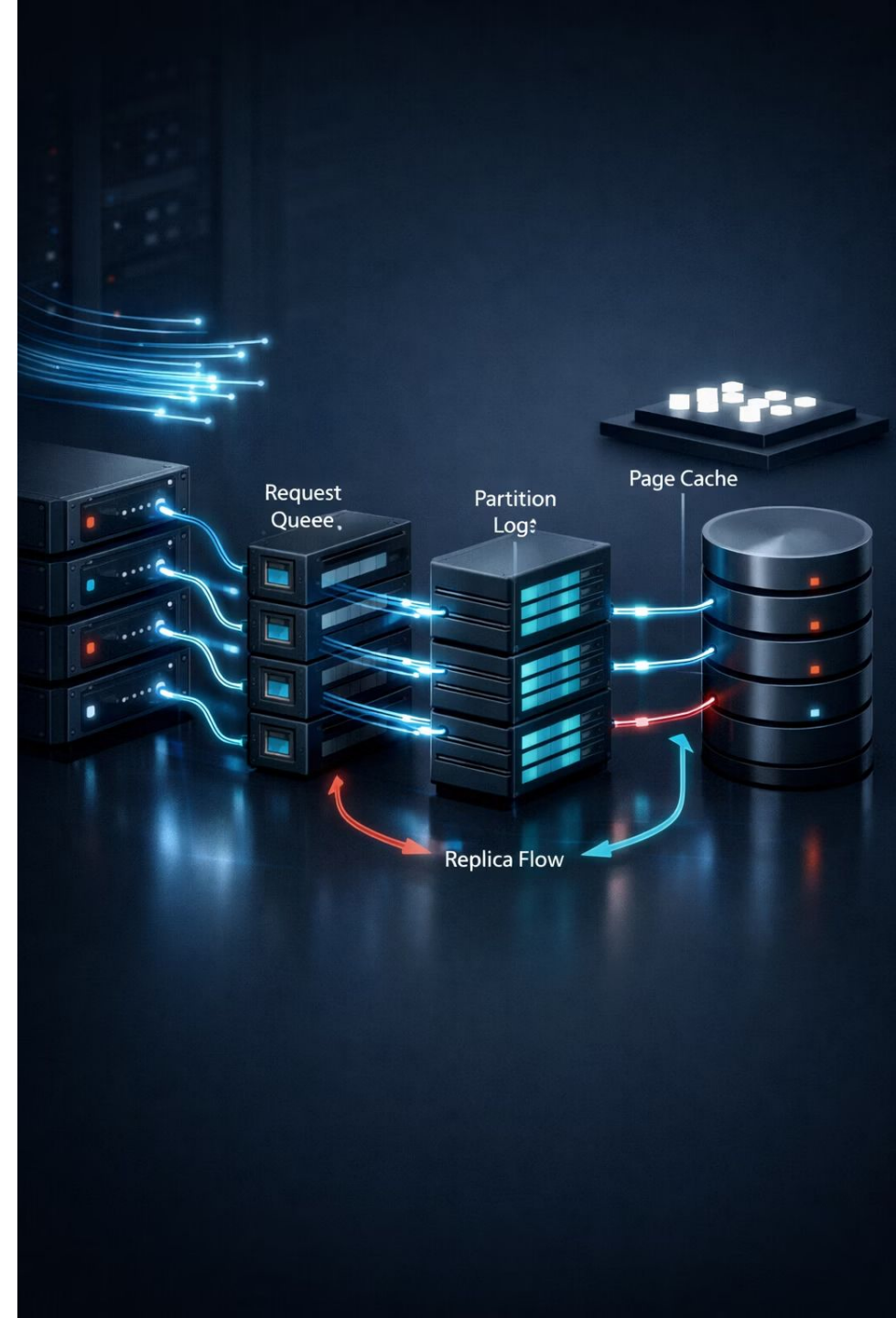
Module 4

Network, Threads and Backpressure

Tuning Kafka's internal thread model and network stack to sustain high throughput while preventing broker overload and request queue saturation.

MODULE 4 OF 7

NETWORK AND REQUEST PROCESSING



num.network.threads: Purpose, Evidence and Trade-off

What It Controls

Number of threads that accept new connections, read bytes from client sockets, and write response bytes back. These threads do no request processing — they are purely I/O handlers.

Evidence for Increasing

NetworkProcessorAvgIdlePercent metric falling below 30%. High connection count (thousands of clients). Slow SSL handshake rate limiting accept throughput.

Benefit

More threads handle concurrent socket I/O without blocking each other. Each network thread manages multiple connections via Java NIO selectors.

Trade-off and Risk

More threads consume OS thread stack memory and context-switch overhead. Candidate: 4–8 for high-connection brokers. Workload testing required. No restart required (per-broker dynamic).

num.io.threads: Purpose, Evidence and Trade-off

What It Controls

Number of threads that dequeue requests from the request queue, validate records, interact with the log subsystem, coordinate with the replica manager, and generate responses.

Evidence for Increasing

RequestHandlerAvgIdlePercent metric below 20%.
Request queue depth growing under normal load.
Produce p99 latency rising without corresponding disk I/O increase.

Benefit

Parallelises log writes and response generation across more CPU cores. Critical for brokers with many active partition leaders.

Trade-off and Risk

I/O threads compete for page cache and disk bandwidth. Excessive thread count causes lock contention on the log. Candidate: 8-16 threads. Workload testing required. No restart required (per-broker dynamic).

queued.max.requests and Broker Backpressure

How the Queue Provides Backpressure

When the request queue reaches `queued.max.requests`, the network threads stop reading from client sockets. TCP receive buffers fill up. The TCP stack applies backpressure to the client via flow control. Clients experience increased latency rather than dropped connections.

Tuning Considerations

- Default: 500 requests. Controls the maximum number of queued requests waiting for an I/O thread.
- Evidence for increasing: Queue consistently near 500 during peak with healthy I/O idle ratio
- Risk of increasing: Higher memory use; more requests in flight on broker crash; masks I/O thread undersizing
- Investigate I/O thread count first before raising this value. No restart required.

Socket Buffers, NIC Bandwidth and Connection Pressure

Parameter	Controls	Candidate Value	Restart?
socket.send.buffer.bytes	TCP SO_SNDBUF per connection; controls outbound data buffering	1 MB (workload dependent)	Yes
socket.receive.buffer.bytes	TCP SO_RCVBUF per connection; controls inbound data buffering	1 MB (workload dependent)	Yes
max.connections	Maximum simultaneous client connections per broker	Set per tenant quota policy	No

i Administrator takeaway: On 10 GbE or 25 GbE NICs, default socket buffers (100 KB) can underutilise available bandwidth on high-latency network paths. Set -1 to allow OS auto-tuning via tcp_rmem/tcp_wmem, which is often the best choice.

Compression and Batching Effects

→ Producer-Side Batching (linger.ms / batch.size)

Larger producer batches amortise per-request overhead across more records, reducing network round-trips and increasing broker I/O efficiency. The broker receives fewer, larger requests — reducing I/O thread invocations per MB of data.

→ Compression on the Broker

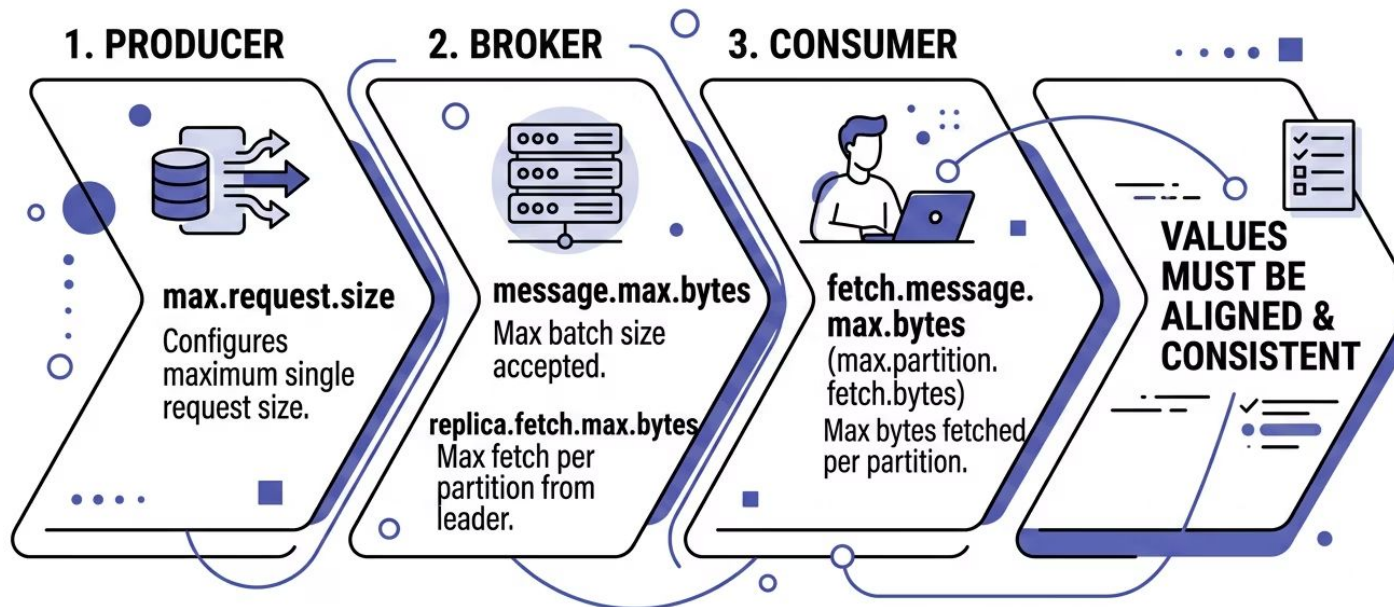
If the broker's `compression.type` for a topic differs from the producer's, the broker recompresses every batch. This is CPU-intensive and adds significant I/O thread latency. Best practice: set `topic compression.type=producer` to pass compressed batches through unmodified.

→ Message Decompression for Validation

The broker decompresses batches to validate record contents and check offsets. This CPU cost is unavoidable. LZ4 and Snappy offer the best decompression throughput; ZSTD offers the best compression ratio with moderate CPU.

Message-Size Settings Across Producer, Broker, Consumer and Replication

Message size limits must be configured consistently across four locations. A mismatch causes producers to receive `RecordTooLargeException` or consumers to receive fetch errors.



⚠ Always increase all four settings together when raising message size limits. Increasing broker `message.max.bytes` without raising `replica.fetch.max.bytes` causes replication failures for large messages.

Connection Limits and Tenant Quotas

`max.connections.per.ip`

Limits connections from a single IP address. Useful for preventing a misconfigured client from exhausting broker connection pool. Default: unlimited. Apply per-broker dynamically.

Producer/Consumer Byte-Rate Quotas

Set via `kafka-configs.sh` with `--entity-type` clients. Controls how many bytes per second a client is permitted to produce or consume. Broker throttles excess via delay injection without disconnecting the client.

Request-Rate Quotas

Limits the fraction of I/O and network thread time a client principal may consume. Prevents noisy-neighbour tenants from starving critical producers on a shared cluster.

Network/Request Tuning Decision Table

Use this table to select the correct parameter based on the observed symptom. Apply changes on a canary broker and validate before promoting cluster-wide.

Observed Symptom	Key Metric	Primary Parameter	Restart?
Slow connection acceptance	NetworkProcessorAvgIdlePercent < 30%	num.network.threads ↑	No
High produce latency, healthy disk	RequestHandlerAvgIdlePercent < 20%	num.io.threads ↑	No
Broker rejecting large messages	RecordTooLargeException in producer logs	message.max.bytes ↑ (all four aligned)	Yes
Tenant consuming all broker capacity	Other clients throttled unexpectedly	Client byte-rate quota ↓	No

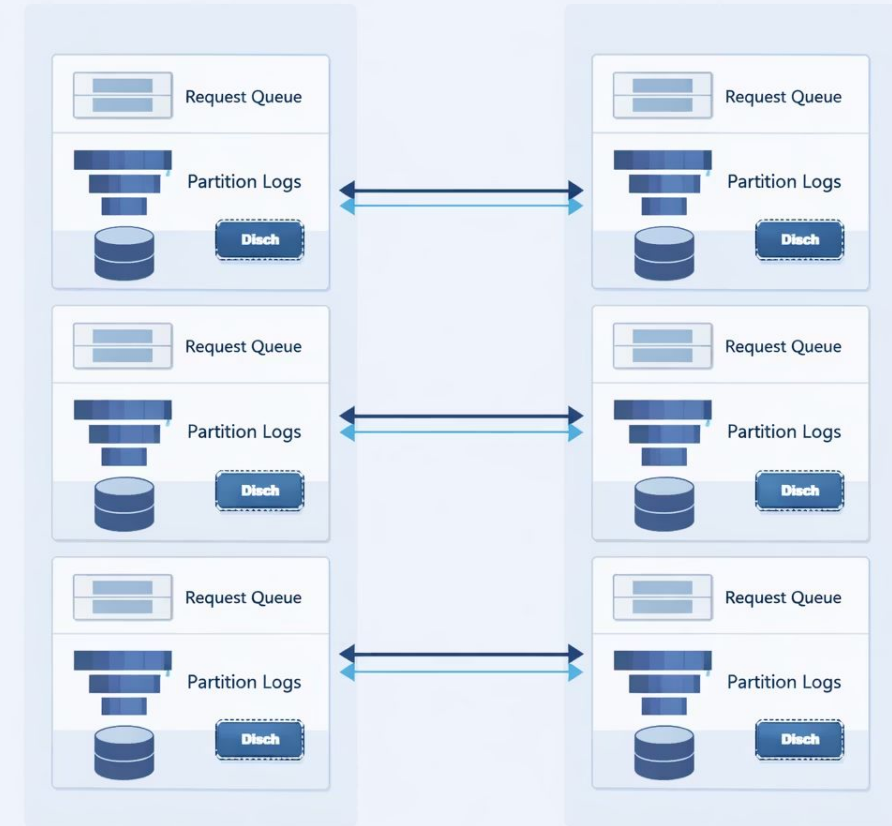
Module 5

Replication, Availability and Recovery

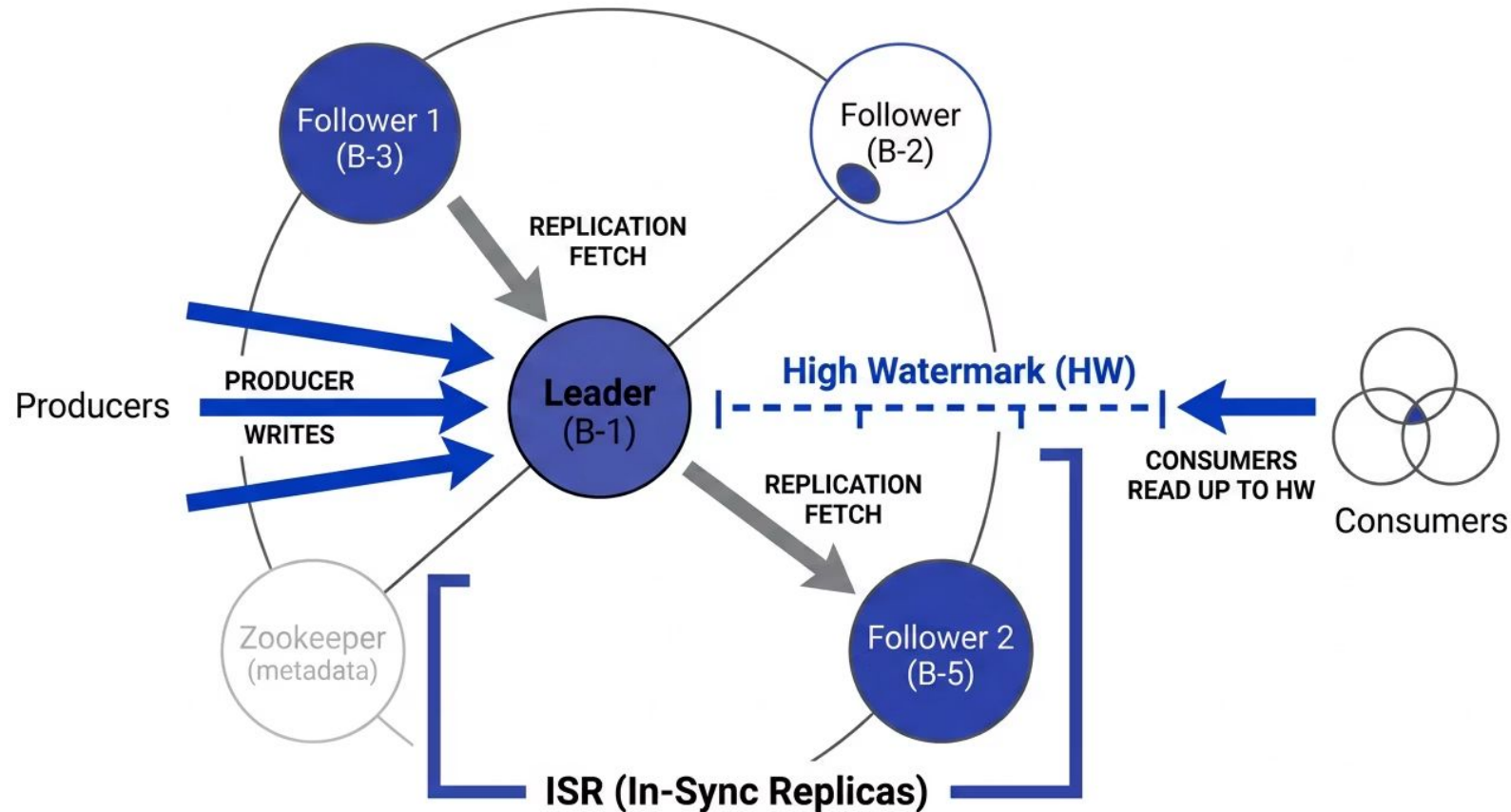
How Kafka replication maintains data safety, how to detect and respond to ISR shrink events, and how to recover from replica lag and slow followers.

MODULE 5 OF 7

REPLICATION AND RECOVERY



Leader, Follower, ISR and High Watermark



The High Watermark (HW) is the maximum offset a consumer can read. It advances only when all In-Sync Replicas have fetched the offset. This design ensures consumers never read data that has not been safely replicated, preventing data divergence after leader failover.

num.replica.fetchers and Follower Catch-Up

What It Controls

Number of `ReplicaFetcherThread` instances per follower broker. Each thread fetches from one leader broker. A follower with partitions on many leaders needs more fetcher threads to keep all replicas in sync simultaneously.

Evidence for Increasing

Replica lag growing on a follower despite healthy disk and network. `ReplicaFetcherManager` metrics showing threads at capacity. ISR shrink events on brokers with high partition counts.

Benefit

Parallel fetching from multiple leaders prevents single-thread bottleneck when a broker holds follower replicas for thousands of partitions across many leaders.

Trade-off and Risk

Each fetcher thread consumes network bandwidth and CPU. Candidate: increase from default 1 to 4–8 on high-partition-count brokers. Workload testing required. No restart required.

Replica Lag, ISR Shrink and Root Causes

Root Cause	Evidence	Likely Indicator Metric
Slow disk on follower	Follower log end offset not advancing; high disk await on follower host	kafka.server:type=ReplicaManager, UnderReplicatedPartitions
Network congestion between brokers	Replication fetch rate below expected throughput; packet retransmits	BytesInPerSec on inter-broker listener
Too few replica fetcher threads	Lag growing proportionally to partition count	ReplicaFetcherManager:MaxLag metric
GC pause on follower	ISR shrink events correlated with GC stop-the-world pauses in logs	JVM GC pause duration in broker logs

❏ Administrator takeaway: ISR shrink is a symptom, not a root cause. Always identify the underlying cause before increasing `replica.lag.time.max.ms` — masking the symptom worsens data durability risk.

RF=3, min.insync.replicas=2 and acks=all

The Safety Triangle

These three settings work together to define the minimum durability guarantee for every write:

- RF=3: Data exists on three brokers. One broker can fail with no data loss.
- min.insync.replicas=2: Produce requests succeed only when at least two replicas are in-sync. Prevents writes to a single survivor.
- acks=all: Producer waits for all ISR replicas to confirm before declaring success.

Failure Scenarios

- One broker fails → ISR has 2 replicas → writes succeed; durability maintained
- Two brokers fail → ISR has 1 replica → `NotEnoughReplicasException`; cluster correctly refuses unsafe writes
- ISR shrinks to 1 without broker failure → same outcome; investigate lag root cause

❏ Setting min.insync.replicas=1 with RF=3 silently allows writes to a single in-sync replica, eliminating the durability guarantee.

Rack Awareness and Leader Distribution



broker.rack Configuration

Set `broker.rack` to the availability zone identifier (e.g., `az-a`) on each broker. Kafka's rack-aware replica assignment ensures no two replicas of the same partition share a rack, surviving a full AZ outage.



Leader Distribution and Preferred Leaders

Each partition has a preferred leader (the first replica in its assignment list). Over time, manual moves or failures cause leaders to concentrate on fewer brokers. Run `kafka-leader-election.sh` to rebalance preferred leaders periodically.



`auto.leader.rebalance.enable`

When enabled, Kafka automatically triggers preferred leader elections in the background when imbalance exceeds `leader.imbalance.per.broker.percentage`. Monitor for unexpected election events on high-throughput topics.

Reassignment Traffic and Throttling

Partition reassignment copies full segment data between brokers and can saturate inter-broker network bandwidth if unthrottled, degrading producer performance during the operation.

```
# Set reassignment throttle to 50 MB/s on broker 2 (canary first)
```

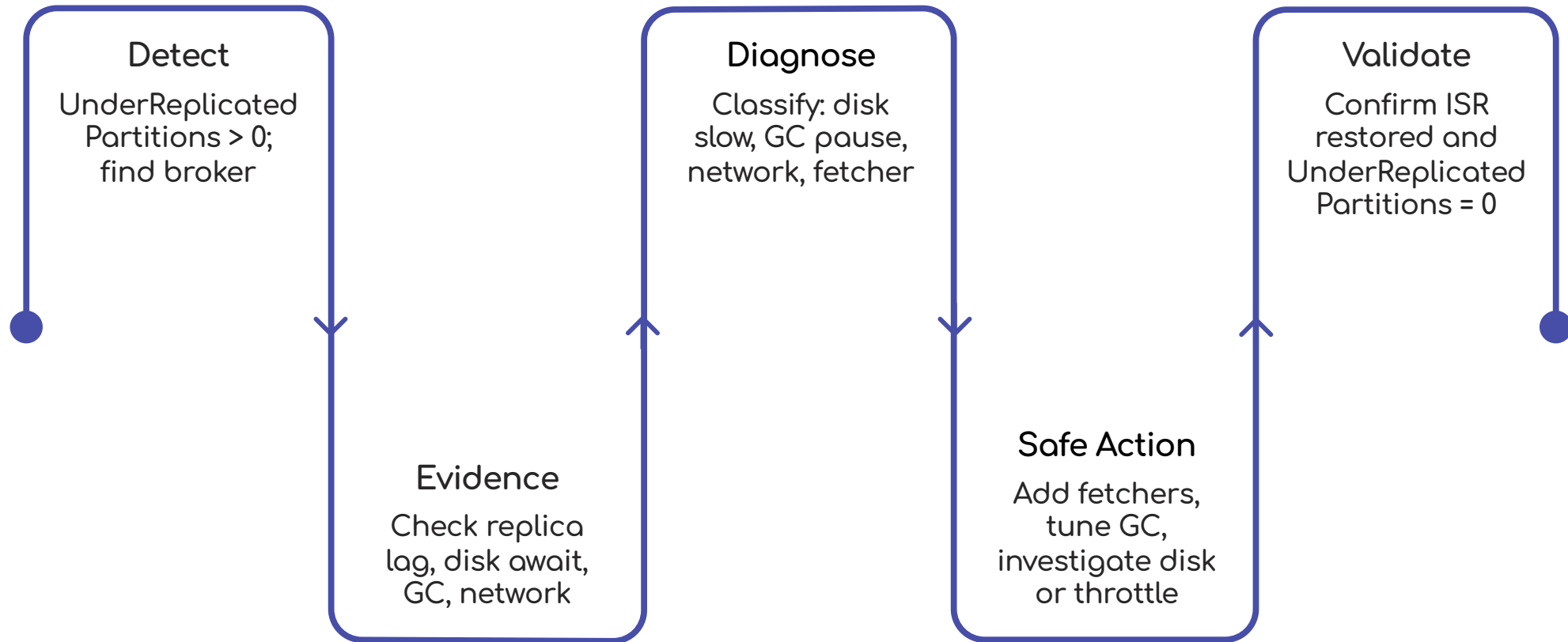
```
kafka-configs.sh \  
  --bootstrap-server broker-1:9092 \  
  --alter \  
  --entity-type brokers \  
  --entity-name 2 \  
  --add-config leader.replication.throttled.rate=52428800,\  
  follower.replication.throttled.rate=52428800
```

```
# Remove throttle after reassignment completes
```

```
kafka-configs.sh \  
  --bootstrap-server broker-1:9092 \  
  --alter \  
  --entity-type brokers \  
  --entity-name 2 \  
  --delete-config leader.replication.throttled.rate,\  
  follower.replication.throttled.rate
```

Always remove throttle configuration after reassignment completes to prevent persistent throughput limitation on production replication traffic.

Slow-Replica Recovery Troubleshooting Flow



Follow this structured flow sequentially. Never skip the evidence step — multiple root causes can present identically. Validate ISR recovery before closing the incident and document findings for the runbook.

Module 6

JVM, Linux and Cloud Runtime

Operating system, JVM, and cloud platform configurations that directly influence broker stability, garbage collection behaviour, and runtime performance.

MODULE 6 OF 7

JVM, OS AND RUNTIME



Heap Sizing vs. Operating-System Page Cache

The Memory Allocation Trade-off

Every byte allocated to JVM heap is a byte unavailable to the OS page cache. Kafka's performance depends overwhelmingly on the page cache, not heap size. A broker with 128 GB RAM should allocate only 6–8 GB to the JVM heap and leave the remainder for the OS.

Heap Sizing Guidelines

- Recommended heap: 6 GB for most production brokers
- Maximum practical heap before GC impact becomes severe: 8 GB
- Set `KAFKA_HEAP_OPTS="-Xms6g -Xmx6g"` — equal Xms and Xmx prevents resize GC events
- Monitor `kafka.server:type=KafkaServer,name=linux-disk-read-bytes` to confirm page cache effectiveness
- Swapping is the most common JVM-related performance disaster — disable swap entirely on Kafka hosts

Garbage Collection, File Descriptors, Swap and Recovery Threads

Garbage Collector Selection

Use G1GC with `MaxGCPauseMillis=20ms`. ZGC is a viable alternative for very low-latency workloads. Avoid CMS (deprecated). GC pause logs should be enabled (`-Xlog:gc*`) and monitored for stop-the-world events exceeding 200 ms, which correlate with ISR shrink.

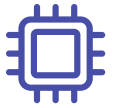
File Descriptor Limits

Each log segment requires three file handles (`.log`, `.index`, `.timeindex`). A broker with 2,000 partitions and 20 segments each needs 120,000 file descriptors. Set OS `ulimit -n 1000000`. Verify via `/proc/<pid>/limits`.

`num.recovery.threads.per.data.dir`

Controls threads used for log recovery on startup after an unclean shutdown. Increase to reduce restart time on brokers with large numbers of segment files. Workload testing required; higher values consume more CPU during startup.

Containers and Cloud: CPU Limits, Memory Limits and Storage Risks



CPU Throttling in Containers

cgroups CPU limits can throttle Kafka broker threads even when the host has idle capacity. GC pauses extend dramatically under CPU throttle. Prefer CPU *requests* over hard *limits* for Kafka containers, or use guaranteed QoS class in Kubernetes.



Memory Limits and OOM Risk

Container memory limits that include page cache space can cause the OOM killer to terminate the broker process unexpectedly. Set container limits above JVM heap + off-heap + expected hot page-cache working set. Monitor container memory usage continuously.



Persistent Storage in Cloud

Network-attached block storage (EBS, Persistent Disk) adds I/O latency vs. local NVMe. Performance is bounded by provisioned IOPS, not disk physics. Size provisioned IOPS based on peak produce throughput, not average. Avoid multi-attach volumes for Kafka log directories.



Module 7

Tuning Method, Scenarios and Labs

A disciplined, evidence-based workflow for applying and validating configuration changes — plus hands-on lab scenarios drawn from real production incidents.

MODULE 7 OF 7

TUNING WORKFLOW AND PRACTICE

Safe Tuning Workflow: Baseline → Hypothesis → Canary → Compare → Rollback

1

Baseline

Record current values of all relevant metrics for one complete traffic cycle (minimum 24 hours). Document the configuration state exactly.

2

Hypothesis

State the specific metric you expect to improve, the parameter you will change, and the candidate value. One parameter per change cycle.

3

Canary

Apply the change dynamically on one broker only (e.g., broker-3). Allow one full traffic cycle before evaluating.

4

Compare

Measure the target metric on the canary broker versus control brokers under identical traffic. Statistical significance required before declaring improvement.

5

Rollback or Promote

If the hypothesis is confirmed, promote to remaining brokers sequentially. If not confirmed or if a regression is detected, use `--delete-config` to revert immediately.

Hands-On Labs, Golden Metrics, Go-Live Checklist and Key Takeaways

Three Lab Scenarios

Lab A: High Produce Latency

Symptom: p99 produce latency exceeds SLA. Investigate I/O thread idle, queue depth, and disk await. Apply and validate `num.io.threads` increase on canary broker.

Lab B: Uncontrolled Disk Growth

Symptom: Broker disk filling faster than capacity plan. Set `log.retention.bytes`, verify segment roll interval, confirm retention check frequency.

Lab C: Recurring ISR Shrink

Symptom: `UnderReplicatedPartitions` alerts multiple times per day. Trace through evidence: GC logs, disk await, network throughput, fetcher thread count.

Official References

- Apache Kafka 4.3 Documentation: kafka.apache.org/documentation
- Broker Configs: kafka.apache.org/documentation/#brokerconfigs
- Operations Guide: kafka.apache.org/documentation/#operations

Golden Metrics to Monitor Always

- `RequestHandlerAvgIdlePercent` — I/O thread headroom
- `NetworkProcessorAvgIdlePercent` — network thread headroom
- `UnderReplicatedPartitions` — replication health
- `ActiveControllerCount` — must equal 1 at all times
- `OfflinePartitionsCount` — must equal 0 at all times
- `BytesInPerSec` / `BytesOutPerSec` — throughput baseline
- `LogFlushRateAndTimeMs` — flush behaviour

Key Takeaways

- Measure before changing — one parameter at a time
- Concepts precede configuration — understand the mechanism first
- Defaults are a starting point, not a recommendation
- `RF=3 + MIR=2 + acks=all` is the minimum safe production baseline
- The page cache is Kafka's most important performance resource